

Authentication and Access Control

Task 1 :- Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

ANS :-

```
<!DOCTYPE html>
<html>
<body style="font-family: Arial, sans-serif; text-align: center; margin-top: 50px;">
  <h2>Zomato Admin Login</h2>
  <input type="text" id="user" placeholder="Username"><br><br>
  <input type="password" id="pass" placeholder="Password"><br><br>
  <button onclick="login()">Login</button>
  <h3 id="msg"></h3>

  <script>
    function login() {
      const u = document.getElementById('user').value;
      const p = document.getElementById('pass').value;
      const msg = document.getElementById('msg');

      if (u === "admin" && p === "zomato123") {
        msg.style.color = "green";
        msg.innerText = "Access Granted";
      } else {
        msg.style.color = "red";
        msg.innerText = "Access Denied";
      }
    }
  </script>
</body>
</html>
```

- Username : admin
- Password : Zomato 1234

Task 2 :- Write a Python function `check_access(user_role, resource)` that returns `True` if the `user_role` is allowed to access the resource, otherwise `False`. Test it with roles like `'customer'`, `'restaurant_owner'`, and resources like `'menu_edit'`, `'order_history'`, `'admin_dashboard'`.

ANS :-

```
# Permissions setup
permissions = {
    'admin': ['menu_edit', 'order_history', 'admin_dashboard'],
    'restaurant_owner': ['menu_edit', 'order_history'],
    'customer': ['order_history']
}

def check_access(user_role, resource):
    # Check if role exists and has access to the resource
    return resource in permissions.get(user_role, [])

# Testing the function
print(check_access('customer', 'order_history'))    # True
print(check_access('customer', 'menu_edit'))        # False
print(check_access('restaurant_owner', 'menu_edit')) # True
print(check_access('restaurant_owner', 'admin_dashboard')) # False
```

Task 3 :- Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

ANS :-

```
import random

# Correct password
PASSWORD = "password123"

# Step 1: Check password
user_pass = input("Enter password: ")

if user_pass == PASSWORD:
    # Step 2: Generate & show OTP
    otp = str(random.randint(100000, 999999))
    print(f"Your OTP is: {otp}")

    # Step 3: Verify OTP
    user_otp = input("Enter OTP: ")

    if user_otp == otp:
        print("Login Successful")
    else:
        print("Wrong OTP!")
else:
    print("Wrong password!")
```

Task 4 :- Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.

ANS :-

```
function getActivePremiumUsers(users) {
  const sevenDaysAgo = new Date();
  sevenDaysAgo.setDate(sevenDaysAgo.getDate() - 7);

  return users.filter(user =>
    user.isPremium && new Date(user.lastLogin) >= sevenDaysAgo
  );
}

// Quick Test:
const users = [
  { username: "Alex", isPremium: true, lastLogin: "2026-10-02" }, // Active Premium
  { username: "Sam", isPremium: false, lastLogin: "2026-10-01" }, // Not Premium (False)
  { username: "Riya", isPremium: true, lastLogin: "2026-08-01" } // Old Login (False)
];

console.log(getActivePremiumUsers(users));
```

Task 5 :- Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

ANS :-

- No login check — Always verify the user is logged in.
- Wrong user access — Check the resource belongs to that user.
- Admin access for everyone — Give only the permissions each role needs.
- Guessing user IDs — Never trust IDs from the request; verify access.
- Missing checks on APIs — Protect every API endpoint, not just the frontend.